Statistics 101C Final Project

Predicting NBA Game Outcomes from Historical Data

Aidan Perez(305830341), Danielle DeFrancisci (905940412), Michael Batrakov (506179471), Parsa Mirpour (806230185), Yash Shahani (006285209), & Yinqi Yao(406290015)

Department of Statistics and Data Science, UCLA

Contents

1 Introduction..... 2
2 Data Preprocessing..... 2
2.1 Initial Preparation..... 2
2.2 Weighted Averages..... 3
2.3 Specifying matchup differences..... 3
2.4 Adding a “Home Advantage” column..... 3
2.5 Specific Feature Engineering..... 4
2.6 Continuous Retraining Approach..... 5
3 Experimental Setup..... 5
3.1 Logistic Regression..... 5
3.2 LinearSVC..... 6
3.3 XGBoost..... 7
3.4 Random Forest..... 7
3.5 Continuous Retraining for Logistic and Random Forest..... 8
4 Analysis and Results..... 8
4.1 Logistic regression..... 8
4.2 Linear Support Vector Classifier..... 9
4.3 XGBoost..... 10
4.4 Random Forest..... 11
4.5 Logistic regression and Random Forest with continuous retraining..... 12
5 Conclusions..... 13

1 Introduction

In the following paper, we detail our analysis of historical NBA data and its ability to predict the game outcome. Our dataset holds statistics on each of the 2023-2024 NBA regular season games. The features of the dataset include team, match-up, game date, a win/loss indicator, minutes played, points scored, field goals made, field goals attempted, field goal percentage, three-point field goals made, three-point field goals attempted, three-point field goal percentage, free throws made, free throws attempted, free throw percentage, offensive rebounds, defensive rebounds, total rebounds, assists, steals, blocks, turnovers, personal fouls, and the plus/minus statistic.

For our analysis, we trained the following models: Logistic Regression, Linear Support Machine Vectors (LinearSVC), and Random Forest. Out of these three models Logistic Regression was the strongest despite clear downfalls to the methodology. The following sections will detail the preprocessing steps, experimental design, and analysis it took to reach that conclusion.

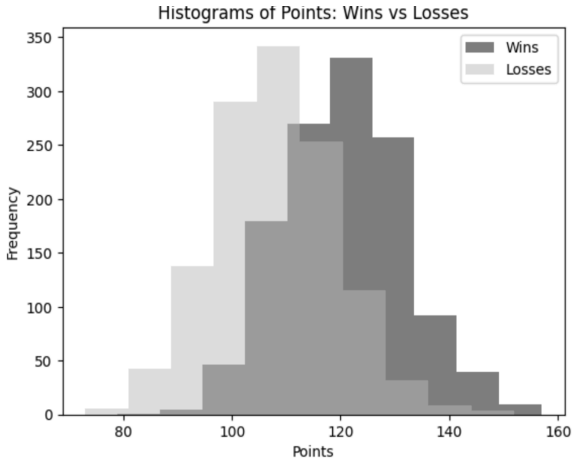
2 Data Preprocessing

We went through several data preprocessing steps to prepare our models for training. After loading the data with the readxl library, we focused on initial feature analysis, addressing null values, eliminating redundant variables, adding additional features, weighting average statistics, splitting the data, and variable selection.

2.1 Initial Preparation

We first worked to address the null values in the dataset. Because there was only one NA value in the entire dataset, null values were not a huge issue. To address it we replaced the null value with the mean of its column. This implementation strategy was chosen since there was only one null value so it was an easy choice. We next checked that the data was sorted in ascending order to ensure chronological consistency in our analysis. Finally, we did some initial plotting to look at potentially strong features. One that was notable was the points column which had distinct distributions for the two potential game outcomes.

Figure 1: Imbalanced points distribution for game outcomes



The distinction seen in this histogram is an early indication of points being a strong feature in our modeling. While there is definitely overlap between the wins and losses histograms, the distinct peaks of each are promising and show that the methods of averaging are logical.

2.2 Weighted Averages

Our initial weighting process went as follows as for each team we computed the weighted average of their statistics using a time-sensitive function:

$$w_i = e^{i/n}$$

Utilizing row numbers to assign heavier weights to more recent games, we fit the training data by team, calculated their weighted statistics, and stored them in a separate weighted_stats dataframe. Using these weighted averages we created splits of 80% training and 20% testing data. However, due to underperformance of our model we circled back to our weighting formula testing other formulas such as:

$$w_i = \alpha(1 - \alpha)^i$$

The best performing formula was not found until much later with an exponential decay function with i indicating the order and n indicating total games per team:

$$w_i = 1.05^{i/n}$$

$$w = \frac{w}{\sum_{i=1}^{n} w_i}$$

For this formula to work we also made sure that the weights were normalized to sum to 1. We also calculated the differences between the two teams for each weighted average statistic to ensure our ability to compare the two teams.

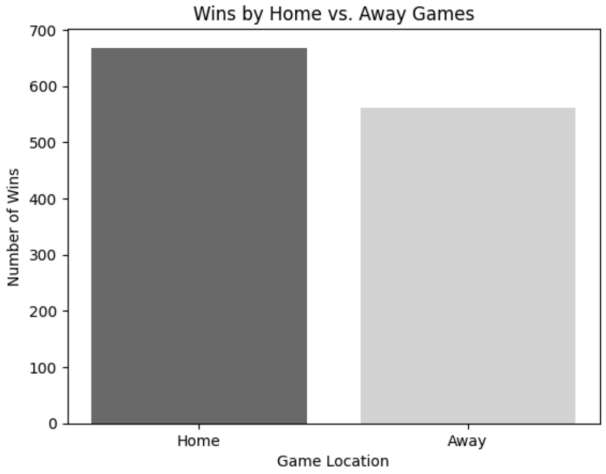
2.3 Specifying matchup differences

For each game we extracted the matchup teams (e.g., GSW vs. PHX) and retrieved the weighted average of each statistic for each team and computed their differences. We saved these computed differences in “stats.differences.train” and “stats.differences.test” in order to create a target “label” column for game outcomes. Computing the differences allows us to spot any gaps in performance between teams within a game. This type of metric can work to highlight which areas one team may hold advantage in over their opponent which could be crucial in our ability to predict game outcomes.

2.4 Adding a “Home Advantage” column

Since it is expected that a team performs better at home compared to on the road, we first looked to see if our data would match this intuition with the plot below.

Figure 2: Imbalance of game outcomes by game location



Based on this plot, itt was clear that we needed to add a home advantage column. Utilizing the text data in the “Match Up” column we were able to create a binary variable for home advantage. By checking the column value to detect either a “vs” or an “@” we were able to assign the binary values with 1 being home and 0 being away. We added these values to the newly created “stats.differences.train” and “stats.differences.test” to help specify the matchups further.

2.5 Specific Feature Engineering

We next decided to eliminate the “Team” column from our dataset. The “Team” column was unnecessary due to the same information being found in the “Match Up” column which we utilized instead. We also removed the “Game Date” column since we had already verified that the games were in chronological order therefore making this column redundant.

Table 1: Summary of feature engineering steps

Original Column	Preprocessing Actions	Final Column
Team	Eliminated	
Match Up	Eliminated, but extracted Team 1 and Team 2 to compute differences of ability from the two teams	
	Created from detecting “vs” in Match Up Column	Home Advantage
Game Date	Eliminated (Data still in chronological order)	
W/L	Created as target for game outcomes (Binary)	Label (Response)
W/L	Subtract Team 1 W/L Weighted Mean and Team 2 W/L Weighted Mean	W/L
MIN	Subtract Team 1 MIN Weighted Mean and Team 2 MIN Weighted Mean	MIN
PTS	Subtract Team 1 PTS Weighted Mean and Team 2 PTS Weighted Mean	PTS
FGA	Subtract Team 1 FGA Weighted Mean and Team 2 FGA Weighted Mean	FGA
FG%	Subtract Team 1 FG% Weighted Mean and Team 2 FG% Weighted Mean	FGP
3PM	Subtract Team 1 3PM Weighted Mean and Team 2 3PM Weighted Mean	TPM
3PA	Subtract Team 1 3PA Weighted Mean and Team 2 3PA Weighted Mean	TPA
3P%	Subtract Team 1 3P% Weighted Mean and Team 2 3P% Weighted Mean	TPP
FTA	Subtract Team 1 FTA Weighted Mean and Team 2 FTA Weighted Mean	FTA
FTM	Subtract Team 1 FTM Weighted Mean and Team 2 FTM Weighted Mean	FTM

FT%	Subtract Team 1 FT% Weighted Mean and Team 2 FT% Weighted Mean	FTP
OREB	Subtract Team 1 OREB Weighted Mean and Team 2 OREB Weighted Mean	OREB
DREB	Subtract Team 1 DREB Weighted Mean and Team 2 DREB Weighted Mean	DREB
REB	Subtract Team 1 REB Weighted Mean and Team 2 REB Weighted Mean	REB
AST	Subtract Team 1 AST Weighted Mean and Team 2 AST Weighted Mean	AST
STL	Subtract Team 1 STL Weighted Mean and Team 2 STL Weighted Mean	STL
BLK	Subtract Team 1 BLK Weighted Mean and Team 2 BLK Weighted Mean	BLK
TOV	Subtract Team 1 TOV Weighted Mean and Team 2 TOV Weighted Mean	TOV
PF	Subtract Team 1 PF Weighted Mean and Team 2 PF Weighted Mean	PF
+/-	Subtract Team 1 +/- Weighted Mean and Team 2 +/- Weighted Mean	PM

2.6 Continuous Retraining Approach

After continued struggles with accuracy, we took a new approach to the data preprocessing for our logistic and random forest models. In order to improve our accuracy we worked through a more dynamic approach that implements a dynamic data preprocessing and modeling approach to predict game outcomes using logistic regression and random forest models. Weighted team statistics are initialized using the first 20% of the dataset, with more recent games assigned exponentially higher weights. From 20% to 80% of the dataset, the differences in weighted statistics for each matchup, along with a home advantage indicator, are saved as training examples. After the 80th percentile, models are retrained iteratively for each game, and predictions are made, with the results incorporated back into the training data to continuously refine the models. This approach achieved predictive accuracies of 74.24% for logistic regression and 74.31% for random forest, demonstrating the effectiveness of iterative retraining and dynamic weighting in capturing team performance trends.

3 Experimental Setup

3.1 Logistic Regression

Logistic regression was the first model implemented in our analysis to predict basketball game outcomes. Despite its limitations, it served as an essential starting point for exploring the relationships between engineered features and game results. Logistic regression's simplicity and interpretability made it a logical choice for initial modeling, but its drawbacks became evident as we evaluated its performance.

To implement logistic regression, it was necessary to first compute the weighted average statistics for each team using only the training dataset. This approach aligns with the predictive nature of the analysis, as it avoids incorporating information from future game outcomes, which are the target of prediction. Weighted averages of each team's game statistics were computed using an exponential decay function, where recent games had greater influence:

$$w_i = 1.05^{i/n}$$

$$w = \frac{w}{\sum_{i=1}^{n} w_i}$$

where i is the game's chronological order, and n is the total number of games for the team. We then normalized each of the weights to sum to 1. Differences between the weighted average statistics of the two teams in a matchup were calculated for all numeric features, capturing the comparative performance between opponents.

The training and testing datasets were split chronologically, with 80% of older games used for training and 20% of newer games reserved for testing. This temporal split ensured the model was tested on unseen and more recent matchups.

The logistic regression model was trained using the glm function in R, with the binary response variable (W/L) representing game outcomes. The predictors included all engineered features: the differences in weighted statistics of competing teams and an additional binary home advantage variable. To refine the model, three approaches were explored: stepwise selection using Akaike Information Criterion (AIC), Lasso regression, and Ridge regression.

Three logistic regression models were tested in this analysis: one utilizing stepwise selection based on Akaike Information Criterion (AIC), another employing L1 regularization (Lasso), and the third applying L2 regularization (Ridge). Stepwise selection aimed to identify the most influential predictors by iteratively adding or removing variables to optimize model fit. Lasso regression introduced sparsity by penalizing the absolute values of coefficients, effectively selecting a subset of the most important features and setting others to zero. Ridge regression, in contrast, penalized the squared values of coefficients, shrinking less significant predictors toward zero without entirely excluding them. Cross-validation was employed to determine optimal penalty parameters for both Lasso and Ridge regularization techniques, which were $\lambda_{lasso} \approx 0.01$ and $\lambda_{ridge} \approx 0.02$. Among the three logistic regression models, the Ridge regression model demonstrated the best performance, achieving a training accuracy of 66.82% and a testing accuracy of 69.31%. This result underscores Ridge regression's ability to balance model complexity and generalizability effectively.

However, the logistic regression model demonstrated significant limitations. Despite the inclusion of multiple engineered features, only two variables—win/loss percentage (W/L) and home advantage—were consistently identified as significant predictors. This limited the model's ability to capture the full complexity of basketball game outcomes, which often involve nonlinear interactions and subtle patterns between team statistics.

The regression-based nature of logistic regression further restricted its performance. By assuming a linear relationship between the predictors and the log-odds of the response variable, the model was unable to account for non-linear relationships inherent in the data. This limitation, combined with its reliance on a single decision boundary, made logistic regression less suitable for this problem compared to ensemble methods like Random Forest or XGBoost.

3.2 LinearSVC

The next model we looked into was Linear Support Vector Machines (LinearSVC). This model was selected since the dataset seemed to exhibit linear patterns which could benefit from a linear decision boundary method. Initially, we tried a model with the same preprocessing data as we utilized for logistic

regression. We then looked at K-fold cross validation and hyperparameter tuning with a goal to prevent overfitting.

Before training our LinearSVC model we addressed the need for feature scaling by applying the ‘train’ function, knowing SVM models are sensitive to this. We then split the data into five folds for cross validation and utilized ‘traincotrl’ to ensure consistent and robust processes. Next, we used the ‘tuneGrid’ parameter to test multiple values of the regularization parameter. After conducting a grid search for the regularization parameter *C* for 0.1 increment values, this led to a suggested value of C to be 0.1 which we then utilized in the next steps. This optimal value provides a balance in its ability to minimize classification errors and maximize margin. The final model utilized this calculated value of C while leaving the other parameters unchanged.

3.3 XGBoost

We also decided to evaluate the predictive power of this data using the XGBoost model. As a decision tree-based learning algorithm, we believed its ability to handle complex feature interactions and non-linear relationships would enable it to catch patterns that our previous linear models could not.

After preprocessing the data similar to our previous two models, we converted the features and labels into specialized xgb.DMatrix objects. We then used a 5-fold cross validation grid search to find the best hyperparameter combinations, which we identified to be 50 boosting rounds, a maximum tree depth of 3, a learning rate of 0.1, a subsample ratio of .8, and minimum child weight of 1. Our final model was trained using this combination of hyperparameters with the objective function to predict the binary output (win/loss) of each game.

3.4 Random Forest

The final model we utilized to predict the basketball game outcomes was random forest. Random forest is a model that works to average decorrelated trees which reduces correlation in return for a slight increase of bias and variance. For this particular data set, random forest may be influential due to its ability to handle non-linear relationships. A potential downside to this model is its ability to overfit the data.

The experimental set up for the random forest modeling created a robust model with 500 trees to ensure prediction stability. At first, each split had five predictors randomly selected in order to prevent decorrelation between trees (mtry = 5). The initial selection of these hyperparameters was later adjusted so that the number of splits was reduced to three to reduce overfitting. We also set the importance metric to true in order to output an evaluation showing which features contributed to the model the greatest. After processing we utilized thresholding for our binary predictors and worked to compare predicted labels with actual labels in the column. Additionally we looked at feature importance with multiple indices providing insights into key features.

3.5 Continuous Retraining for Logistic and Random Forest

For methods of logistic regression and Random forest, we altered our processes in order to achieve higher accuracy. The continuous retaining model approach follows a pattern with its hyperparameters ‘load’ and ‘split’. The ‘load’ variable denotes the beginning point in the dataset to collect the weighted averages of the team statistics data frame. For our models we utilized the first 20% of the data solely to establish the weighted averages of the team statistics. The ‘split’ parameter denotes the point of the dataset where it is split into training and testing distinctions. We maintained an 80% training to 20% testing split of the data throughout this process to maintain consistency in our methods. After the 80% point of the data, we would continuously retrain the random forest and logistic regression models and predict unseen game matchups. This data would then be added to the training set, and we would update the weighted team statistics for the specific teams in the matchup.

4 Analysis and Results

4.1 Logistic regression

The results of the logistic regression models highlight their performance and limitations in predicting basketball game outcomes. Three variations of logistic regression were tested: stepwise selection using Akaike Information Criterion (AIC), L1 regularization (Lasso), and L2 regularization (Ridge). Each model's performance was evaluated using training and testing datasets, and the results are summarized in the table below.

Table 2: Training and testing performance across logistic models before continuous retraining

Model	Training Accuracy	Testing Accuracy
Logistic Regression with Stepwise AIC	66.62%	69.72%
Logistic Regression with L1 Regularization (Lasso)	66.92%	68.50%
Logistic Regression with L2 Regularization (Ridge)	66.82%	69.31%

Logistic regression with stepwise AIC achieved a training accuracy of 66.62% and a testing accuracy of 69.72%. The stepwise selection process helped identify the most influential predictors, refining the model to its simplest form while maintaining predictive performance. However, its reliance on linear relationships limited the complexity of interactions it could capture, making it less suitable for the intricacies of basketball game data.

Logistic regression enhanced with Lasso regularization, a technique that applies L1 penalization to the model coefficients, achieved a training accuracy of 66.92% and a testing accuracy of 68.50%. By enforcing sparsity, Lasso regularization retained only the most influential predictors, effectively eliminating less significant features. While this approach mitigated overfitting, the observed reduction in

testing accuracy indicates that some excluded features may still hold predictive value, warranting further exploration of feature importance and model optimization.

Logistic regression enhanced with Ridge regularization, a technique that applies L2 penalization to the model coefficients, demonstrated a training accuracy of 66.82% and a testing accuracy of 69.31%. By shrinking coefficients of less significant predictors rather than eliminating them, Ridge provided a balance between feature retention and regularization. This approach yielded the most consistent testing performance among the three models, indicating its effectiveness in handling multicollinearity and stabilizing predictions.

Despite the modest performance of all three logistic regression models, their results reveal significant limitations. Across the models, the reliance on win/loss percentage (W/L) and home advantage as the primary predictors underscored the inability of logistic regression to effectively leverage the full range of engineered features. Furthermore, the assumption of linear relationships between predictors and the log-odds of the response variable restricted the models' capacity to capture nonlinear interactions.

The close alignment of training and testing accuracies across all three models suggests minimal overfitting. However, the overall accuracy levels indicate that logistic regression struggled to capture the full complexity of basketball game outcomes. These findings highlight the need for more advanced models, such as Random Forest or XGBoost, which can better accommodate non-linear relationships and intricate feature dependencies.

While logistic regression served as a foundational approach in this analysis, its limitations in predictive accuracy and feature utilization underscore the necessity of adopting more robust and flexible modeling techniques for this problem.

4.2 Linear Support Vector Classifier

Proceeding with a step up in model complexity, we have our Linear Support Vector Classifier (Linear SVC). After training and testing, this model achieved the following accuracies:

Table 3: Training and testing performance of LinearSVC

Model	Training Accuracy	Testing Accuracy
LinearSVC	67.784%	67.683%

As seen in this table, the performance of our classifier was remarkably consistent across both the training and testing datasets, with the training accuracy exceeding the testing accuracy by only 0.1%. This indicates a well-fitted model with minimal overfitting, which is further supported by the following detailed classification report:

Table 4: Performance Metrics Table of LinearSVC

Class	Precision	Recall	F1-Score
0 (Loss)	.68	.67	.67
1 (Win)	.68	.68	.68
Weighted Avg	.68	.68	.68

The balanced precision and recall across both classes demonstrate the model's main strength: its thoroughness in classifying both wins and losses. The weighted average F1-score of 0.68, which accounts for the nearly equal class distributions, further underscores the model's consistent and reliable performance across both outcomes. As a result, these balanced performance metrics provide sufficient evidence of the model's effectiveness without requiring further breakdowns such as true positive or true negative rates, as the findings from these metrics would be redundant.

Overall, the Linear SVC model demonstrated relatively reliable performance in predicting NBA game outcomes, with testing accuracy on par with the logistic regression model, though not quite as good. Even if it was initially expected to outperform the logistic regression model, the Linear SVC model still shows promise. Nevertheless, its inability to capture non-linear patterns limits its potential for further improvement.

4.3 XGBoost

Despite its complexity, our XGBoost model did not deliver a stronger performance than either of the previous two models discussed. Our training and testing accuracies are as follows:

Table 5: Training and testing performance of XG Boost

Model	Training Accuracy	Testing Accuracy
XG Boost	72.526%	67.480%

As seen in this table, the XGBoost model achieved a training accuracy that was 5% higher than its testing accuracy. While XGBoost is expected to perform better on training data, this discrepancy raises some concerns about potential overfitting. However, despite being larger than the differences observed in the previous models, this gap is not significant enough to conclusively indicate overfitting, particularly due to the extremely flexible nature of the XGBoost model. The classification performance of the model was evalueded further, as seen in the following table:

Table 6: Performance Metrics Table of XG Boost

Class	Precision	Recall	F1-Score
0 (Loss)	.67	.68	.68
1 (Win)	.68	.67	.67
Weighted Avg	.67	.67	.67

The weighted F1-score of 0.67 reflects the model's balanced performance in predicting both outcomes, particularly how there is no bias towards either class. The close alignment between precision and recall across classes, similair to our Linear SVC model, further underscores the model's reliability in identifying both wins and losses.

Additionally, a deeper look into the XGBoost's decision making process shows us the difference in feature importance. The most influential features that had the biggest impact on deciding game outcomes came down to points scored (PTS), rebounds (REB), turnovers (TOV), and home advantage. Unsurprisingly, points scored difference had the highest contribution, reinforcing the obvious fact that teams do not win unless they are scoring lots of points.

Overall, though its results were underwhelming, our XGBoost model at least remained on par with the previous models discussed. More importantly, it provided a deeper insight into feature importance, delineating which historical statistics are more relevant to future performance.

4.4 Random Forest

Our random forest model does not have as high an accuracy as the logistic regression models. However, we believe that with further development, this one would likely be the most logical model to use.

After processing the random forest model we looked at feature importance. We first looked at mean decrease in accuracy (%IncMSE) which shows how the model's accuracy decreases when a particular feature is excluded. We also looked at the mean decrease in Gini Index (IncNodePurity) which produced the same results seen in the table below.

Table 7: Feature Importance

Predictor	With %IncMSE	With IncNodePurity
W/L	36.01	42.98
X	28.18	36.69
X3P	10.97	24.12
FG	10.26	20.13
TOV	9.55	17.58

This table demonstrates that these five features are the strongest so we proceeded to refine our random forest model to solely include these five features. We maintained 500 as the number of trees and shifted the number of predictors selected at each split to three. This however did not increase our accuracy but rather decreased it by a single percentage point. Due to this we may look into refining our preprocessing data further. It is also notable that at this time we had 100% training accuracy indicating overfitting in the model. This led us to readjust to the maximum number of predictors to be selected at each split resulting in the following adjusted values

Table 8: Training and testing performance of Random Forest

Model	Training Accuracy	Testing Accuracy
Random Forest (500, 3)	68.50%	65.65%

*(500, 3) denotes ntree = 500, mtry = 3

This adjusted model has a slightly lower testing accuracy, but it does not have the extreme overfitting as the first model did. Further steps would be needed to work to improve the testing accuracy such as refining parameters nodesize and maxdepth which were left at the default values thus far .

4.5 Logistic regression and Random Forest with continuous retraining

Our shortcomings in accuracy led us to pursue a more dynamic retraining approach. We explored this additional option as a means of experimentation and pursuit of higher accuracy. This method allowed for the statistics to update based on the most recent data available capturing the most real time adjustment as possible. The updated accuracies are listed below, each model maintaining its other parameters.

Table 9: Logistic and Random Forest models with and without continuous retraining

Model	Training Accuracy	Testing Accuracy
Logistic Regression (AIC) without continuous retraining	66.62%	69.72%
Logistic Regression with continuous retraining	-	74.24%
Random Forest (500, 3) without continuous retraining	68.50%	65.65%
Random Forest (500, 3) with continuous retraining	-	74.31%

The only model adjustments made were utilizing a more basic form of logistic regression, but all hyperparameters were maintained. The continuous retraining model is only able to produce testing accuracy due to continuous development of its nature. Still, we see a large increase in testing accuracy with this process being implemented.

There are still limitations to this additional process. It is a much more computationally extensive approach due to the consistent updating of the model. It is also not a standard test and train model which could also lead to possible overfitting. The higher accuracy of this model indicates promising results and a hope to explore this process further with the potential inclusion of more features and refinement.

5 Conclusions

This project aimed to predict NBA game outcomes using data from the 2023 NBA season by experimenting with various machine learning models including Logistic Regression, LinearSVC, XGboost, and random forest. Despite its inherent limitations in capturing complex relationships, logistic regression achieved the highest testing accuracy among the models explored, outperforming both LinearSVC and Random Forest. Throughout the model refining process it became apparent that this project requires significantly more feature engineering and preprocessing, however, each additional feature we experimented with was eliminated during standard feature selection processes. Moving forward we would work to possibly adjust the weighting of the statistics to create a more accurate adjustment for each model, and additionally continue to experiment with different features.

Logistic regression, particularly Ridge regression (L2 regularization), demonstrated the highest testing accuracy among the standard models explored. However, its linear assumptions limited its ability to capture the non-linear dynamics of basketball game outcomes. LinearSVC achieved 67.68% testing accuracy but failed to outperform logistic regression, even with hyperparameter tuning. XGboost achieved a similar testing accuracy and hinted at potential overfitting, despite proving to be a more complex model. Random Forest initially struggled with overfitting, indicated by 100% testing accuracy in its first stages, and even though refinements improved its performance, it remained less accurate than logistic regression.

The unexpected success of logistic regression underscores the importance of effective feature engineering. Weighted averages, home advantage, and chronological data splitting highlighted critical predictors but may have constrained the performance of more complex models. Future work should explore richer feature sets, advanced ensemble methods like XGBoost, and better handling of home and away splits to enhance predictive accuracy.

Moving into the adaptive models with logistic regression and random forest, they both outperformed the standard models and performed almost identically to one another. These clearly were more suited to the task of the data at hand, however, they were much more computationally extensive and did not follow standard modeling procedures.

While these dynamic models delivered the strongest results, their complexity highlights the potential of more advanced models to capture the full scope of the basketball data. The work and insights gained from this project act as a strong foundation for further development and research.